SilenceLink: A Large Multimodal Model–Driven Application for Hong Kong Sign Language Translation

Technical Summary

Wai-Shing Ng
Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
wsngam@connect.ust.hk

September 2026

Scope of this summary

This technical summary was prepared by Wai-Shing Ng from a team final-year project. It presents the system architecture, engineering decisions, training strategy, evaluation approach, and deployment design in a concise format suitable for academic or interview discussion.

The summary intentionally omits licensed source video, dataset files, annotations, raw qualitative examples, model checkpoints, and third-party source code. All diagrams, keypoint thumbnails, gloss labels, and sentence examples shown in this document are synthetic or self-authored explanatory material. This document describes a team project and does not claim that all system components were implemented by the summary author.

Project motivation and objectives

SilenceLink is a research prototype for Hong Kong Sign Language (HKSL) translation and bidirectional communication. The project investigates whether a large multimodal pipeline can translate recorded signing motion into written text and optional speech output, while providing a speech-to-text pathway for communication in the opposite direction.

The project had three connected objectives. First, it explored a visual-temporal sign-recognition and translation pipeline for HKSL-oriented inputs. Second, it integrated text-to-speech and speech-to-text extensions to support two-way interaction between sign-language users and non-signers. Third, it deployed the system through a mobile client and server-based inference architecture so that computationally intensive processing could be performed remotely rather than entirely on a mobile device.

The system is a research and engineering prototype rather than a production-ready translator or a replacement for qualified sign-language interpretation.

End-to-end workflow

Figure 1 provides an overview of the complete SilenceLink architecture. To balance computational efficiency with user accessibility, the system is structured into three distinct deployment layers: an LMM inference layer, an API-server layer, and a mobile-application layer.

The LMM inference layer handles the computationally intensive backend tasks, including visual preprocessing via keypoint estimation, dual-stream sign encoding, Connectionist Temporal Classification (CTC) gloss prediction, downstream mBART text generation, and sign-to-speech processing. The API-server layer acts as an intermediary orchestration hub, managing network requests, data transfer, and secure communication between the mobile client and remote inference services. The mobile-application layer serves as the user-facing interface, enabling users to record sign-language videos, capture spoken audio, display translated or recognised text, and play synthesised speech.

For a sign-video translation request, the pipeline extracts structured upper-body and hand keypoints using DWPose, processes them through the dual-stream sign encoder with lateral feature exchange, decodes a predicted gloss sequence, and feeds the output to the mBART sequence-to-sequence model for fluent Cantonese text generation.

The generated text can subsequently be passed to a text-to-speech service to produce spoken audio. For reverse communication, spoken audio is captured by the mobile client, processed by a remote speech-recognition engine, and returned as translated text on the user interface, establishing a seamless two-way interaction pathway.

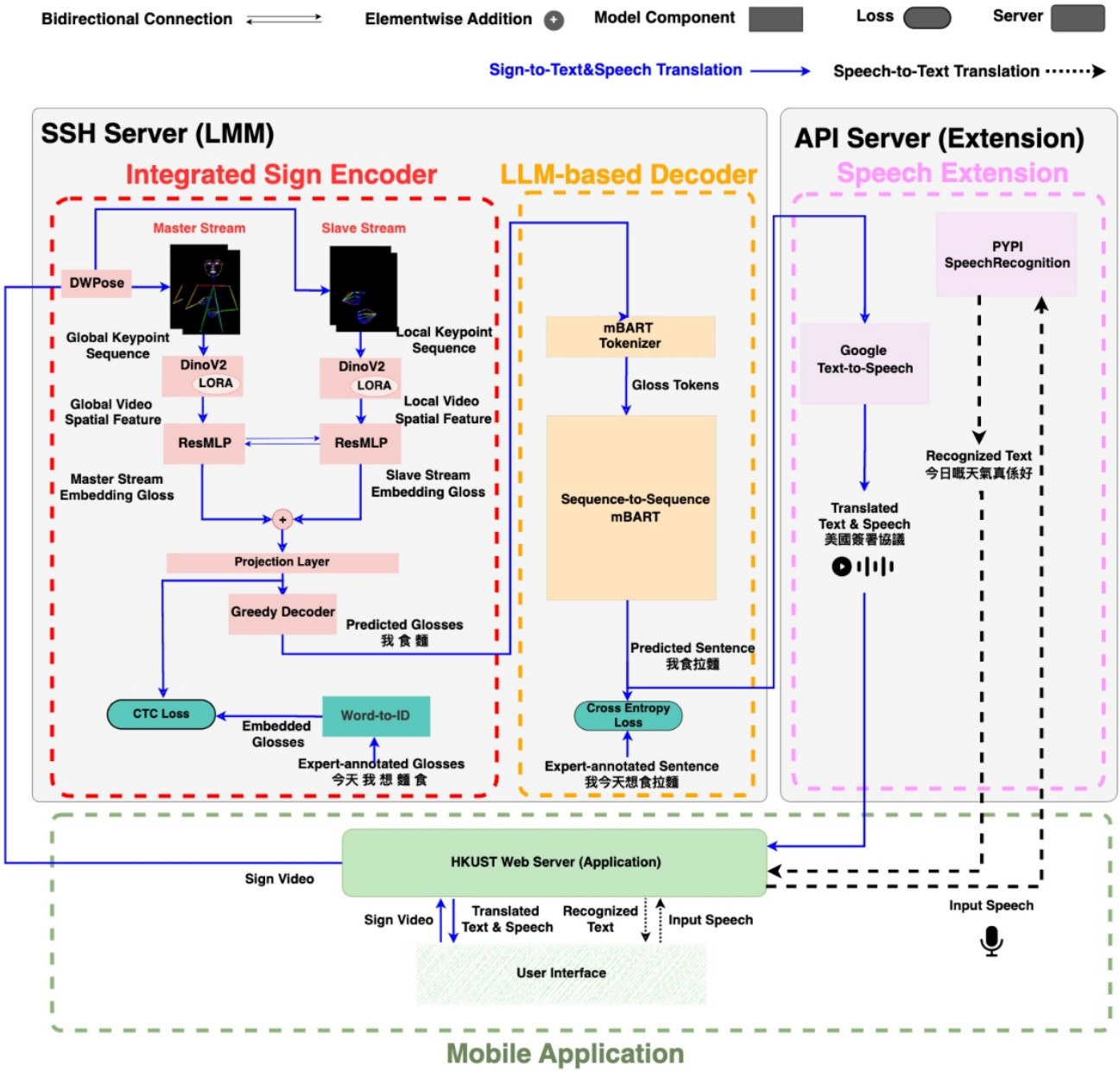


Figure 1: End-to-end SilenceLink workflow. Keypoint thumbnails and visible gloss/sentence labels are synthetic or self-authored illustrative examples; no licensed video frame, dataset annotation, or model output is shown.

Data preparation and preprocessing

The data-preparation stage established the inputs and target representations required by the recognition and translation components. The team verified the availability of associated text labels, excluded incomplete records from the training pipeline, and prepared token representations for the learning tasks.

For sign recognition, a `Word-to-ID` dictionary mapped output gloss tokens to integer indices. This representation supported CTC-based supervision during model training and enabled the inference decoder to map predicted token indices back to a gloss sequence.

For downstream translation, available written labels were prepared in a Cantonese-oriented form for the intended communication setting and segmented into token units. The project also investigated appearance-level augmentation as a way to reduce over-reliance on narrow visual recording conditions. This summary intentionally excludes source frames, augmentation outputs, and dataset-derived images.

DWPose-based keypoint extraction

DWPose preprocessing is the entry point to the sign-recognition path. It processes every input video frame and estimates upper-body and hand-oriented keypoints, converting visual sign motion into structured sequential representations.

The preprocessing stage creates two complementary inputs. The global keypoint sequence captures upper-body posture, arm movement, and broader spatial context. The local keypoint sequence concentrates on hand-oriented movement and fine-grained hand configuration. This structured representation reduces direct reliance on visual background appearance and allows later modules to focus on motion patterns.

The global representation is supplied to the Master Stream, while the local representation is supplied to the Slave Stream. DWPose-based keypoint preprocessing was implemented by a group member and is included to explain the complete team system rather than as the individual work of the summary author.

Spatial model

The spatial model converts each global or local keypoint frame into a high-dimensional spatial feature embedding. These embeddings provide the temporal sign encoder with a representation of posture and motion structure at each point in the sequence.

The project used a pretrained DINOv2-style vision backbone for spatial feature extraction. Pretrained weights allowed the team to reuse useful feature representations instead of optimising the entire visual model from scratch.

To balance adaptation and computational efficiency, most backbone blocks were frozen during fine-tuning. Low-Rank Adaptation (LoRA) modules were introduced into selected later attention blocks. LoRA learns compact low-rank parameter updates rather than updating all pretrained weights, reducing the number of trainable parameters while allowing task-specific adjustment.

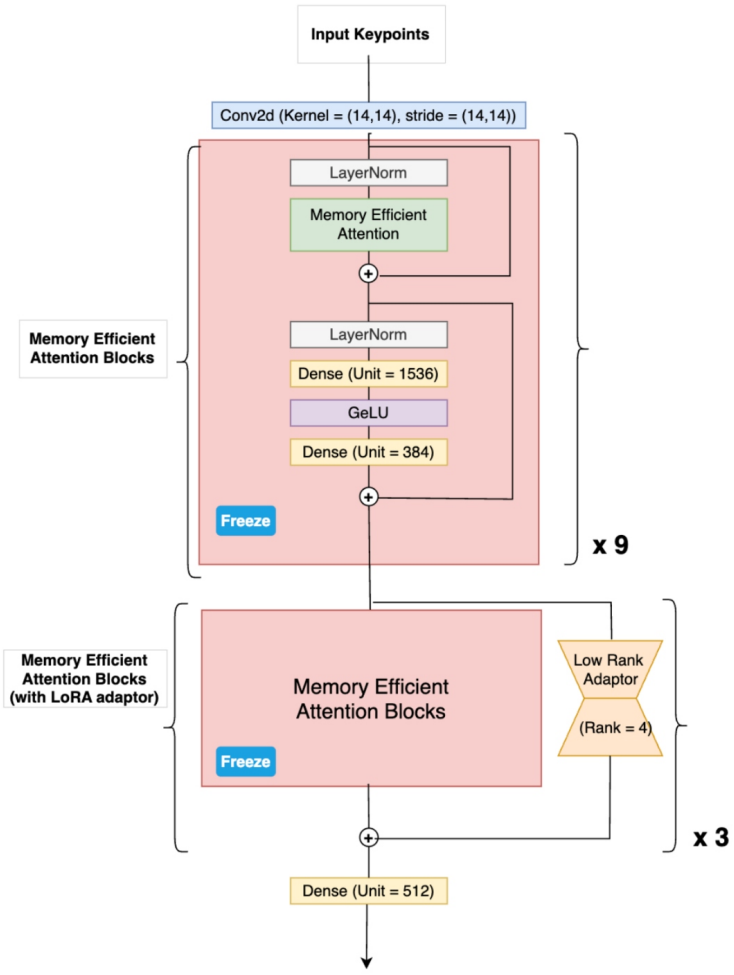


Figure 2: Spatial-model schematic. The pretrained DINOv2-style backbone is largely frozen, while selected later blocks are adapted through LoRA modules.

Temporal sign encoder

The temporal sign encoder receives spatial embeddings from consecutive frames and models the temporal structure of signing motion. Its purpose is to transform frame-level spatial features into representations that support gloss recognition.

The encoder uses stacked ResMLP blocks. The temporal MLP models relationships across time steps, enabling the model to combine evidence from different frames. The channel MLP transforms feature dimensions, while residual paths preserve information flow and support stable optimisation through multiple blocks.

Each block receives a direct input from the preceding block in its own stream. In the dual-stream configuration, each corresponding block can also receive a lateral input from its counterpart in the other stream.

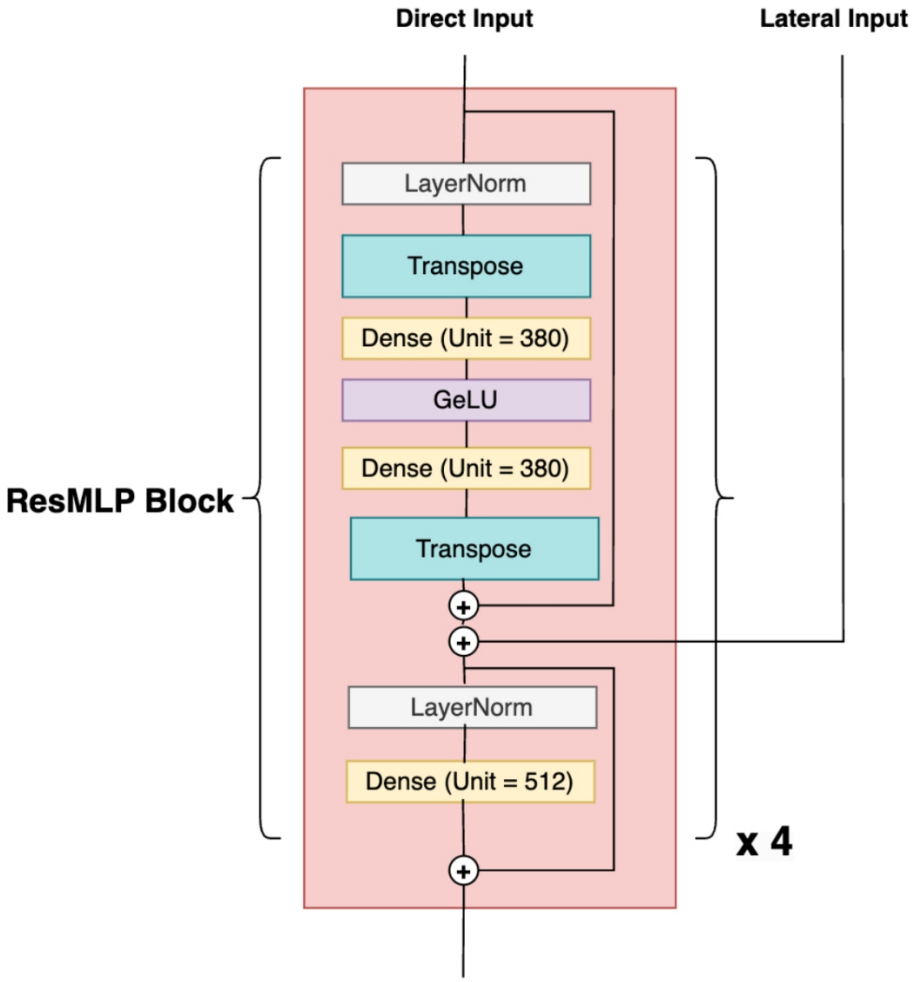


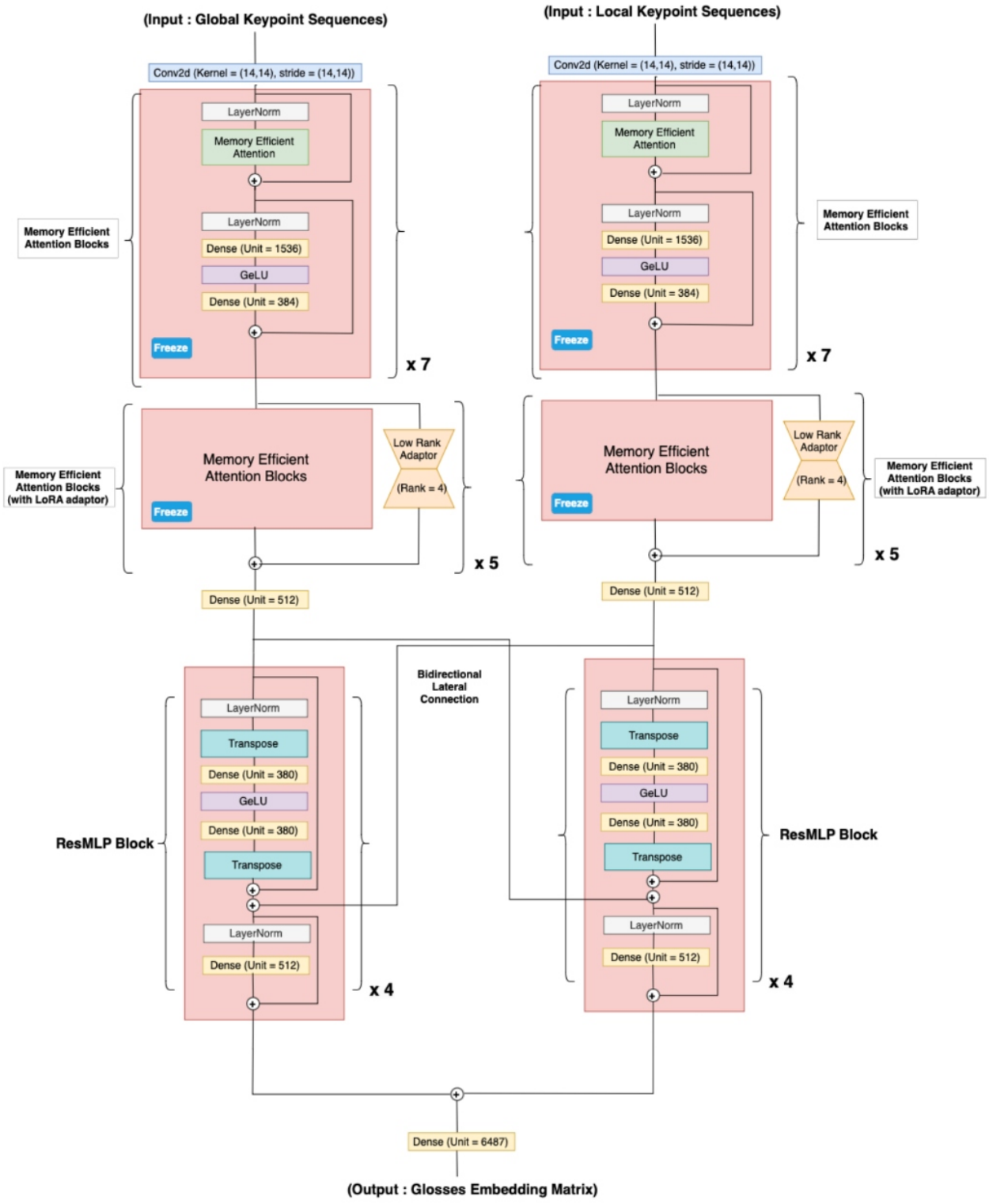
Figure 3: ResMLP sign-encoder block with a direct input from its own stream and a lateral input from the corresponding block in the other stream.

Integrated dual-stream sign encoder

The integrated sign encoder combines spatial feature extraction and temporal modelling in two parallel branches. The Master Stream processes global upper-body keypoint sequences, and the Slave Stream processes local hand-oriented keypoint sequences.

The Master Stream captures broad body context, including posture and arm trajectory. The Slave Stream emphasises local hand movement and configuration. Processing both representations in parallel allows the model to retain global contextual information while attending to hand-level detail.

Each stream contains its own spatial feature extractor and temporal ResMLP encoder. The resulting embeddings are combined by elementwise addition and passed to an output projection layer. The projection layer produces frame-level gloss-token logits for recognition training and decoding.



Integrated Sign Encoder: Dual Stream Model

Figure 4: Integrated dual-stream sign encoder. The Master Stream processes global upper-body keypoint sequences and the Slave Stream processes local hand-oriented keypoint sequences. The streams exchange lateral features and are fused before gloss-token projection.

Bidirectional lateral feature exchange

Bidirectional lateral feature exchange connects corresponding ResMLP blocks in the Master and Slave Streams. This mechanism allows global contextual information and local hand-focused information to influence one another during temporal encoding, rather than being combined only at the final fusion stage.

At the first ResMLP block, each stream receives its own spatial features as the direct input and the other stream's spatial features as the lateral input. At later blocks, each stream receives its preceding output and the preceding output of the corresponding block from the opposite stream.

Let M and S denote the Master and Slave Streams, respectively. Let $\beta \in \{1, \ldots, n\}$ denote the ResMLP-block index, and let $o_{\alpha,\beta}$ denote the output of block β in stream α. Given spatial features Spatial_M and Spatial_S, the simplified explanatory formulation is:

$$
\begin{aligned}
o_{M,1} &= \text{ResMLP}_{M,1}(\text{Spatial}_M, \text{Spatial}_S), \\
o_{S,1} &= \text{ResMLP}_{S,1}(\text{Spatial}_S, \text{Spatial}_M), \\
o_{M,\beta} &= \text{ResMLP}_{M,\beta}(o_{M,\beta-1}, o_{S,\beta-1}), \\
o_{S,\beta} &= \text{ResMLP}_{S,\beta}(o_{S,\beta-1}, o_{M,\beta-1}), \qquad 2 \leq \beta \leq n.
\end{aligned}
$$

This mechanism is distinct from the mobile application's bidirectional communication functionality. Here, bidirectional refers to feature exchange between model streams. At application level, bidirectional communication refers to sign-to-speech and speech-to-text interaction.

Gloss recognition and decoding

The fused dual-stream representation is passed through an output projection layer to produce a probability distribution over gloss tokens for each frame. The integrated sign encoder is trained with a Connectionist Temporal Classification (CTC) objective.

CTC is suitable for sequential recognition because the number of input frames can differ from the number of output gloss tokens. It allows the model to learn an alignment between a variable-length sign-motion sequence and a shorter sequence of symbolic gloss labels.

During inference, greedy decoding selects the highest-probability token at each time step. Consecutive repeated tokens are collapsed and blank tokens are removed. The remaining token indices are mapped through the `Word-to-ID` dictionary to construct the predicted gloss sequence.

Gloss-to-text generation with mBART

The predicted gloss sequence is processed by the downstream mBART decoder to generate written text. mBART is a multilingual sequence-to-sequence model with an encoder–decoder structure suitable for translation tasks.

The mBART tokenizer converts predicted glosses into model-compatible token indices. The decoder then generates a written sequence conditioned on the gloss representation. During inference, beam search explores multiple candidate sequences and selects a high-probability translation instead of relying solely on a single greedy text-generation path.

The recognition and translation components were trained in stages. The integrated sign encoder was first trained for gloss recognition with CTC supervision. The sign encoder was then frozen while the mBART decoder was trained for downstream gloss-to-text translation using a cross-entropy objective.

Speech extensions

The system includes speech extensions to support two-way interaction between sign-language users and non-signers.

For sign-to-speech translation, text generated by mBART is passed to a text-to-speech service. The API layer returns both translated text and synthesised audio to the mobile application.

For speech-to-text translation, the mobile application records spoken input and sends the audio to a speech-recognition service. Recognised text is returned to the user interface. These two paths enable sign-input and speech-input interaction rather than a one-way sign-to-text workflow.

Training strategy and infrastructure

The project trained the sign encoder and downstream text decoder in separate stages. During sign-recognition training, the visual-temporal encoder received keypoint sequences as input and gloss-token sequences as targets. During downstream translation training, the frozen sign encoder produced gloss representations used as input to the mBART decoder.

The training workflow used validation monitoring to select the checkpoint with the best observed validation behaviour. Optimisation used AdamW together with a cosine learning-rate schedule. Gradient accumulation was used to operate under memory constraints while approximating a larger effective batch size.

Training and evaluation used remote GPU-capable infrastructure. The later client–server design reused remote compute for inference, avoiding deployment of the full LMM directly on the mobile device.

API server and mobile application

The mobile application was developed by a group member using React Native with Expo. It provides the user-facing interface for recording sign videos, capturing speech input, displaying translated or recognised text, playing synthesised speech, and navigating between application screens.

The API server is the integration layer between the mobile client and remote processing services. For sign-video requests, it receives the uploaded video, manages request handling and file transfer, invokes the remote inference pipeline, and returns translated text and speech. For speech-input requests, it forwards audio to the speech-recognition service and returns recognised text.

This client–server architecture separates resource-intensive inference from the mobile device. The mobile client manages user interaction and media capture, while remote services perform keypoint extraction, model inference, and speech-service calls.

The mobile application and its application-level integration were developed by a group member. They are described to show how the machine-learning pipeline was deployed as an end-to-end system and are not presented as the individual work of the summary author.

Testing and system validation

The project included component-level and integration-level testing to validate that the model, servers, APIs, and mobile client operated together as intended.

For the model pipeline, testing checked whether the spatial model, temporal sign encoder, gloss-decoding component, and downstream text decoder accepted expected input structures and produced outputs with expected structural properties.

For the mobile application, unit testing covered user-interface behaviour such as navigation, recording-state changes, camera switching, and button interactions. Integration testing examined communication among the mobile client, API server, remote inference environment, and speech services. The testing workflow included request handling, file management, API responses, error handling, and end-to-end inference flow.

Aggregate evaluation results

Table 1 reports aggregate text-generation metrics from the team project's authorised held-out HKSL evaluation setting. It contains no source media, frames, annotations, or individual model predictions.

The strongest reported configuration was the DualStream Skeleton model with bidirectional lateral exchange. It achieved the highest Combined Score among the listed baseline and model-variant configurations, supporting the design motivation for combining global and local keypoint streams with lateral feature exchange.

Table 1: Aggregate evaluation of baseline and team model variants. Combined Score is calculated as BLEU-4 plus ROUGE-2 F1.

Model	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-2 F1	Combined
S3D (Video)	25.39	18.59	14.57	12.10	21.61	33.71
S3D (Skeleton)	19.93	13.72	10.41	8.48	16.42	24.90
SingleStream (Video)	17.30	9.95	4.42	2.50	8.71	11.21
SingleStream (Skeleton)	18.50	11.79	6.14	3.92	9.77	13.69
DualStream (Video, no lateral exchange)	22.10	14.37	8.77	6.31	12.34	18.65
DualStream (Skeleton, no lateral exchange)	23.98	15.88	11.54	8.18	15.13	23.31
DualStream (Video + Skeleton + lateral exchange)	29.23	20.14	17.21	14.70	23.62	38.32
DualStream (Skeleton + lateral exchange)	**31.46**	**22.89**	**19.92**	**17.23**	**25.71**	**42.94**

Limitations and future work

The prototype demonstrates a complete research and deployment pipeline, but it has important limitations. The sign-to-speech path depends on network transfer, keypoint extraction, sign recognition, text generation, and speech synthesis. These stages increase overall latency and limit real-time responsiveness.

The authorised research material had limited diversity in signers and recording conditions. This can reduce generalisation when the system is used with varied signers, backgrounds, lighting conditions, or signing styles. In addition, n-gram-based metrics such as BLEU and ROUGE do not fully measure semantic correctness, linguistic naturalness, or communicative usefulness.

Future work should investigate more diverse appropriately consented data, lightweight pose-estimation approaches, more efficient inference infrastructure, model compression, privacy-preserving handling of video and audio, richer semantic evaluation, and community-informed evaluation with HKSL users and qualified sign-language experts.

Third-party software and attribution

Sign2GPT. Sign2GPT [1] is cited as related work and as an implementation reference only. This technical summary does not include, reproduce, redistribute, or claim ownership of Sign2GPT source code, model checkpoints, configuration files, datasets, or assets.

Any future repository containing copied or modified upstream code should identify the relevant source files and comply with the applicable upstream licence, including retention of required licence and notice material, attribution, and modification notices.

References

[1] R. Wong, N. C. Camgoz, and R. Bowden. Sign2GPT: Leveraging Large Language Models for Gloss-Free Sign Language Translation. In *The Twelfth International Conference on Learning Representations*, 2024.